



Flutter



Sommaire :

- Page 2 : Introduction
- Page 2 : Le langage Dart
- Page 6 : Flutter
- Page 7 : Firebase
- Page 8 : Autorisations
- Page 10 : Les notifications
- Page 11 : Le Système de paiement
- Page 11 : Déploiement sur les stores
- Page 15 : Conclusion

Introduction

Le développement mobile s'effectue aujourd'hui deux façons natives et hybride.

Les langages natives sont Java ,Kotlin pour android et Objectiv-C et swift pour ios.

Les langages ou framework sont Xamarin , react Native et flutter.

Flutter est un kit de développement logiciel (SDK) d'interface utilisateur open-source créé par Google. Il est utilisé pour développer des applications pour Android, iOS, Linux, Mac, Windows, Google Fuchsia et le web à partir d'une seule base de code.

Le langage Dart

Dart est un **langage** de programmation optimisé pour les applications sur plusieurs plateformes. Il est développé par Google et est utilisé pour créer des applications mobiles, de bureau, de serveur et web.

Dart est un **langage** orienté objet à ramasse-miettes avec une syntaxe de type C++. Et est utilisé pour développer des applications sous flutter

Playground dart et Fluttter: <https://dartpad.dev/?>

Nous avons les différents présents en dart comme les chaînes de caractères , les booléens , les nombres décimaux, les list , les dictionnaires, les énumérations etc..

Ex de déclaration de variable

```
late String monChaine ;  
late double monDouble;  
bool monBoolean = true;  
List monTableau =  
late Map<String,dynamic> map;
```

enum énéuration {on,off,auto}

Le type enum est déclaré en dehors d'une classe.

Le terme « late » devant la variable indique que celle sera affecté ultérieurement dans le programme.
Celle-ci est nécessaire pour les variables non optionnelles

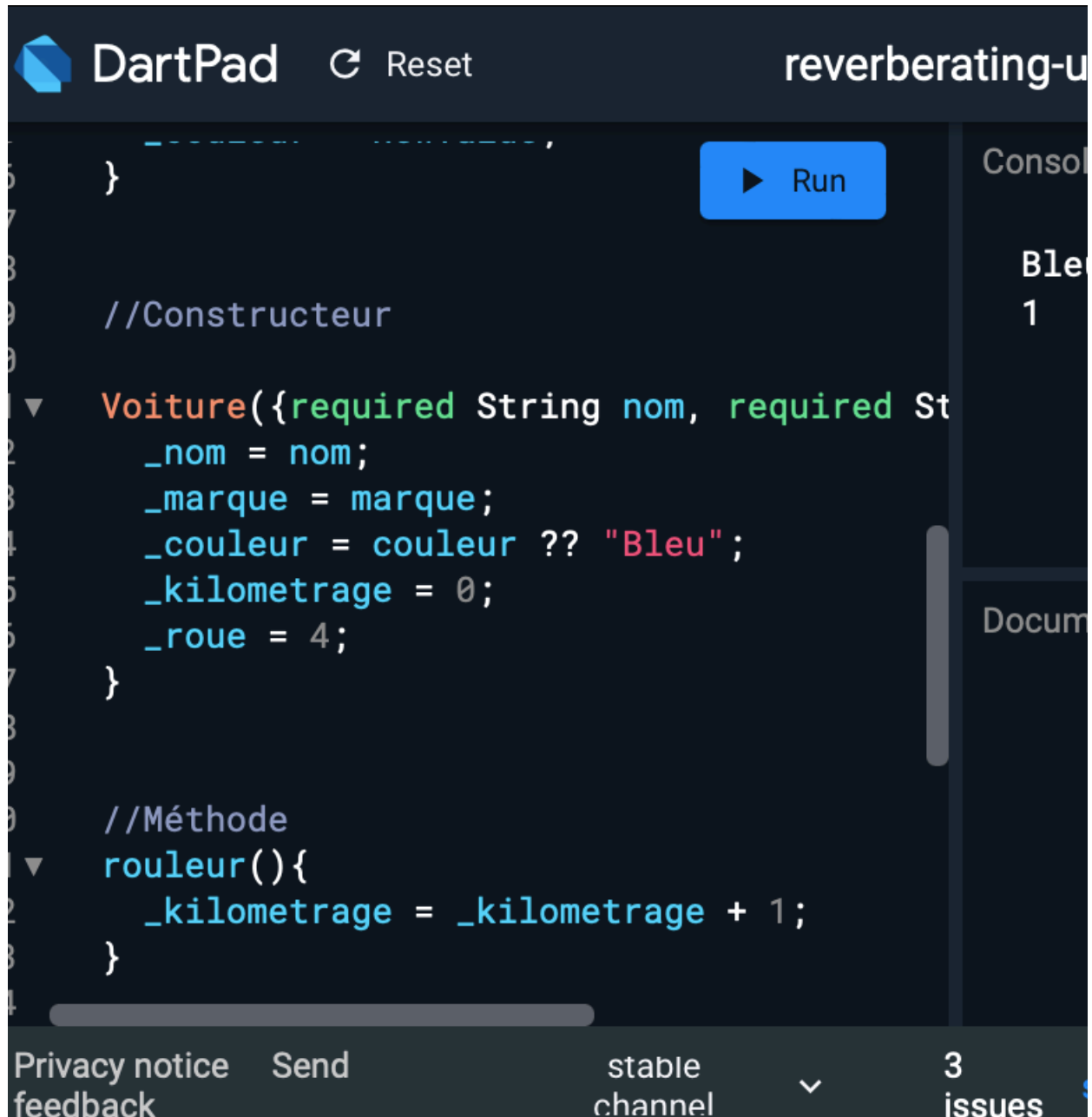
Les optionnels sont établis lorsque la variable est susceptible de prendre la valeur nulle.

La notation est la suivante String? maVariable.
On indique un « ? » juste après le type de la variable.

Lorsque l'optionnel, on souhaite indiquer que l'optionnel est non nulle ou « unwrap » la variable en indiquant un point « ! »

Ex : maVariable!

Dart est un langage orienté objet ainsi il est possible

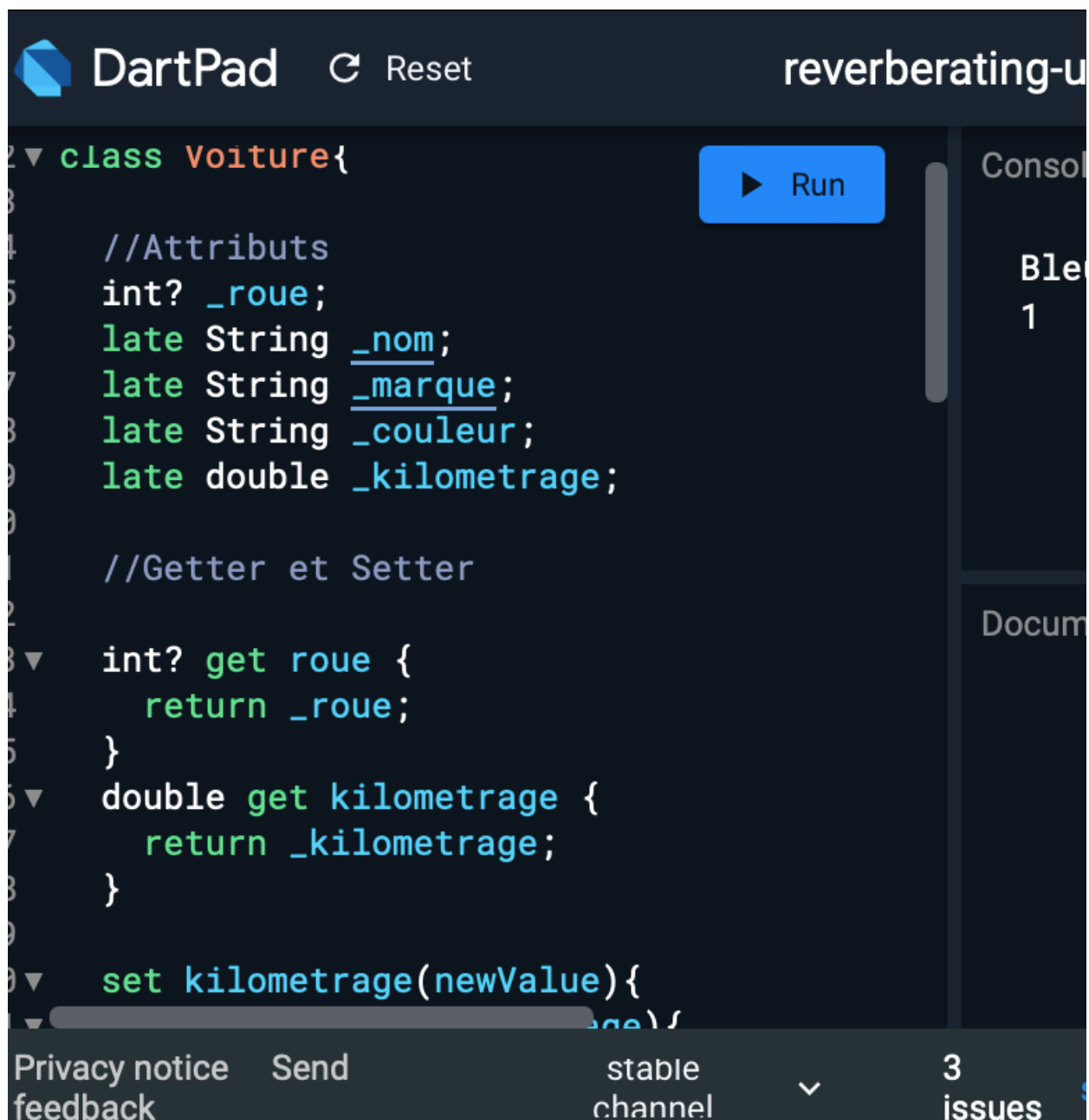


The screenshot shows the DartPad web editor interface. At the top, there's a 'DartPad' logo, a 'Reset' button, and a username 'reverberating-u'. The main area contains Dart code for a 'Voiture' class. The code includes a constructor and a 'rouleur()' method. A 'Run' button is visible in the top right of the code editor. On the right side, there's a 'Console' panel showing the output 'Bleu' and '1'. Below the code editor, there's a 'Privacy notice' link, a 'Send feedback' button, a 'stable channel' dropdown menu, and a '3 issues' indicator.

```
5 }  
6  
7  
8  
9 //Constructeur  
10  
11 ▼ Voiture({required String nom, required St  
12     _nom = nom;  
13     _marque = marque;  
14     _couleur = couleur ?? "Bleu";  
15     _kilometrage = 0;  
16     _roue = 4;  
17 }  
18  
19  
20 //Méthode  
21 ▼ rouleur(){  
22     _kilometrage = _kilometrage + 1;  
23 }  
24
```

de créer des class constitué d'attributs , de méthodes et d'un ou plusieurs constructeurs.

Exemple de Classe sous Dart



The screenshot shows the DartPad web editor interface. At the top, there's a header with the DartPad logo, a 'Reset' button, and the file name 'reverberating-u'. The main editor area displays a Dart class named 'Voiture'. The class has several attributes: an optional integer '_roue', and three late strings '_nom', '_marque', and '_couleur'. It also has a late double attribute '_kilometrage'. Below the attributes, there are comments for getters and setters. The class includes a getter for 'roue' returning '_roue', a getter for 'kilometrage' returning '_kilometrage', and a setter for 'kilometrage' that takes a 'newValue' parameter. A 'Run' button is visible next to the class definition. On the right side, there are panels for 'Console' and 'Documentation'. The bottom status bar shows 'Privacy notice', 'Send feedback', 'stable channel', a dropdown arrow, and '3 issues'.

```
▼ class Voiture{  
    //Attributs  
    int? _roue;  
    late String _nom;  
    late String _marque;  
    late String _couleur;  
    late double _kilometrage;  
  
    //Getter et Setter  
  
    int? get roue {  
        return _roue;  
    }  
    double get kilometrage {  
        return _kilometrage;  
    }  
  
    set kilometrage(newValue){  
        _kilometrage = newValue;  
    }  
}
```

Privacy notice Send feedback stable channel 3 issues

Flutter

Flutter se base sur l'imbrication de Widgets, on peut imaginer de nombreux lego qui s'emboîtent pour pouvoir constituer le Design ou autres.

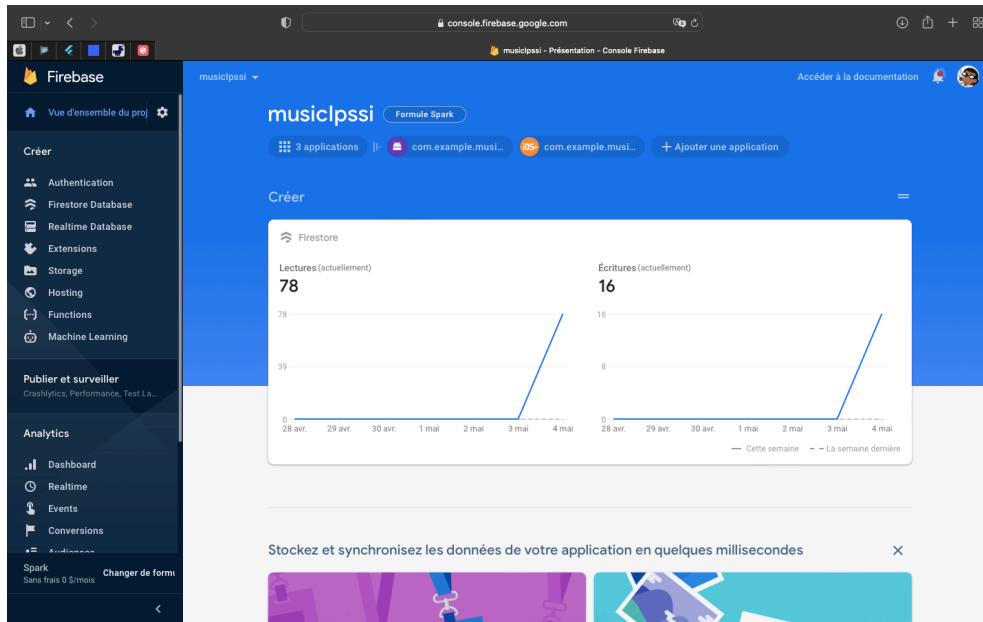
Chaque Widget a des propriétés (appelé attributs)

Voici qu'élues Widgets que l'on peut utiliser dans Flutter tel Container, TextField ,Text ,Image, ElevatedButton, Column , row , Stack.

Cette liste n'est pas exhaustive. Vous pouvez trouver d'autres Widgets intéressant sur le lien suivant :

<https://docs.flutter.dev/development/ui/widgets>

Firestore

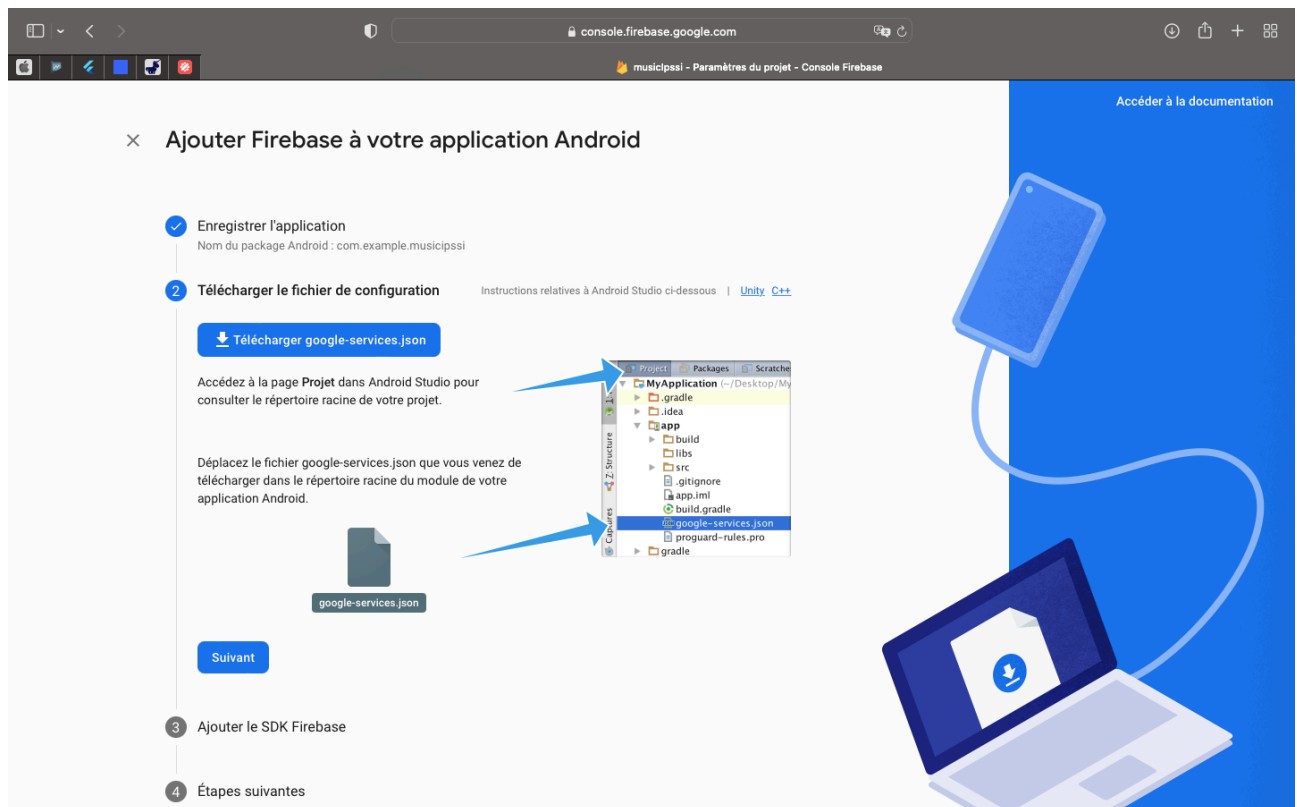


Firestore est un ensemble de service d'hébergement pour n'importe quel type d'application (android, iOS , JavaScript, Node.js, ...)

. Il propose d'héberger en NoSQL et en temps réel des bases de données, du contenu, de l'authentification sociale (Google, Facebook, Twitter et Github), et des notifications, ou encore des services, tel que par exemple un serveur de communication temps réel.

Lors de la création d'une application, il est nécessaire de pouvoir « connecter » notre application à Firestore via la plateforme que nous utilisons (Android et iOS) .Les configurations des systèmes sont différents mais les fichiers GoogleService.json et

GoogleService.plist respectivement pour Android et



iOS sont nécessaires.

Les autorisations

Lorsque votre application souhaite avoir accès à l'utilisation de service où de donnée lié à l'utilisateur . Il est nécessaire de demander les autorisations adéquates.

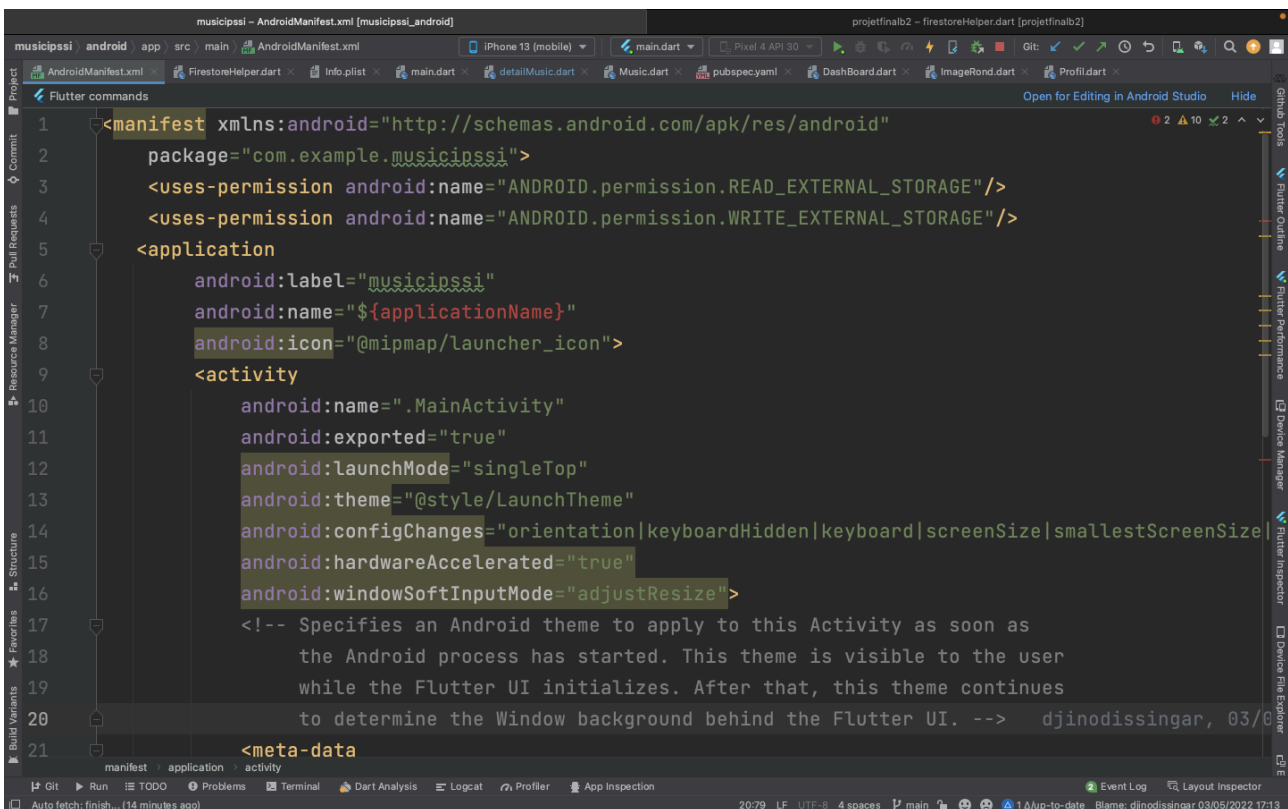
Les autorisations sont intégrés dans le fichier androidManifest.xml et info.plist respectivement pour Android et iOS.

Exemple d'autorisation :

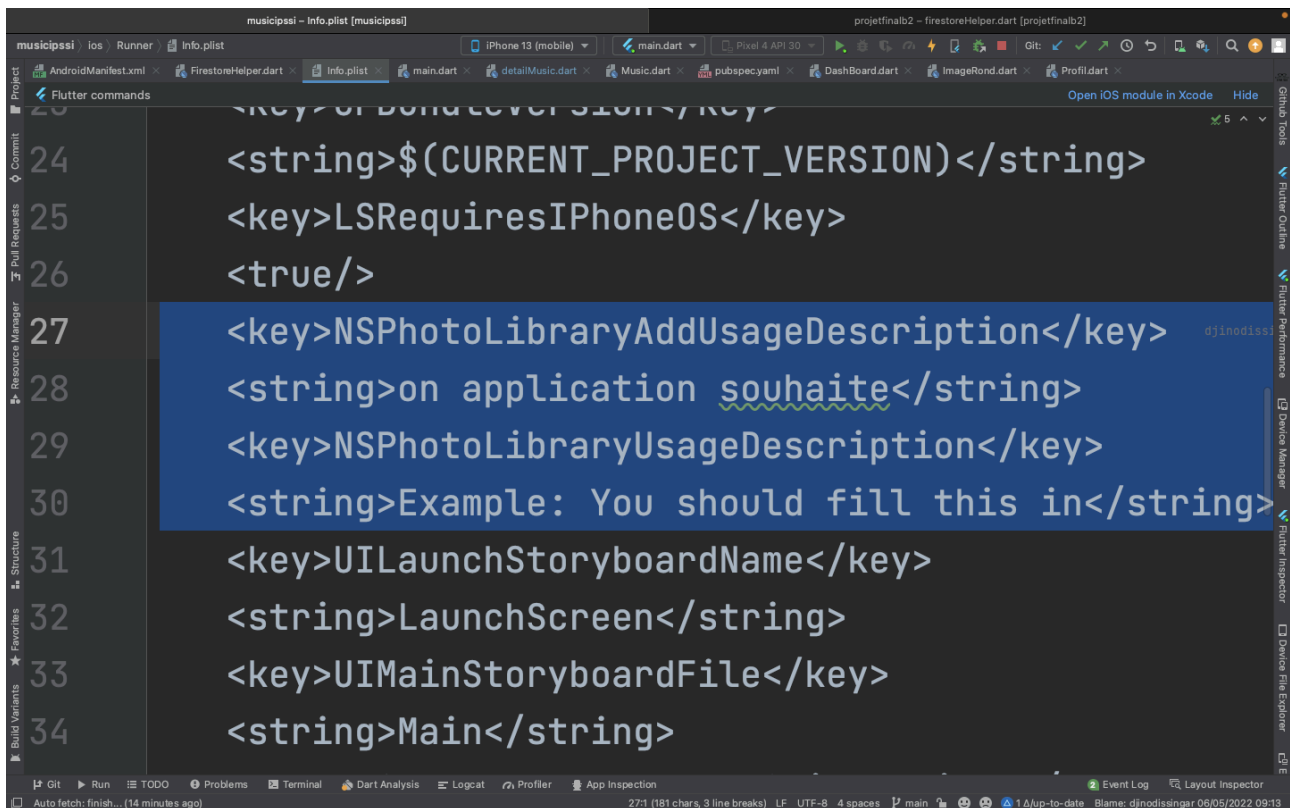
<https://developer.android.com/reference/android/Manifest.permission>

<https://www.iosdev.recipes/info-plist/permissions/>

(Exemple de permission pour Android)



(Exemple de permission pour iOS)



```
24 <string>$(CURRENT_PROJECT_VERSION)</string>
25 <key>LSRequiresIPhoneOS</key>
26 <true/>
27 <key>NSPhotoLibraryAddUsageDescription</key>
28 <string>on application souhaite</string>
29 <key>NSPhotoLibraryUsageDescription</key>
30 <string>Example: You should fill this in</string>
31 <key>UILaunchStoryboardName</key>
32 <string>LaunchScreen</string>
33 <key>UIMainStoryboardFile</key>
34 <string>Main</string>
```

Les Notifications

Il existe deux type de notifications :

1. Les notifications locales celle-ci sont géré par le téléphone de l'utilisateur (ex: rappel dans un calendrier)
2. Les notifications Push (remote notification), les notifications Push sont envoyé via un serveur à l'appareil distant cela se concrétise souvent par les notifications de promotion envoyé par des société ou certaines informations (ex : lorsque votre colis a été réceptionné)

Le système de paiement

Il existe plusieurs façons pour rémunérer son application :

1. La rémunération peut se faire via de la publicité
2. La rémunération peut être également via l'application , en intégrant un système de paiement (ex : Stripe , PayPal). Ce type de rémunération ne peut s'effectuer que pour des produits matériel , pour les produits immatériels ou de service par abonnement, cela est de l'achat in-app et il est obligatoire de passer par le système de paiement de Apple ou Google.

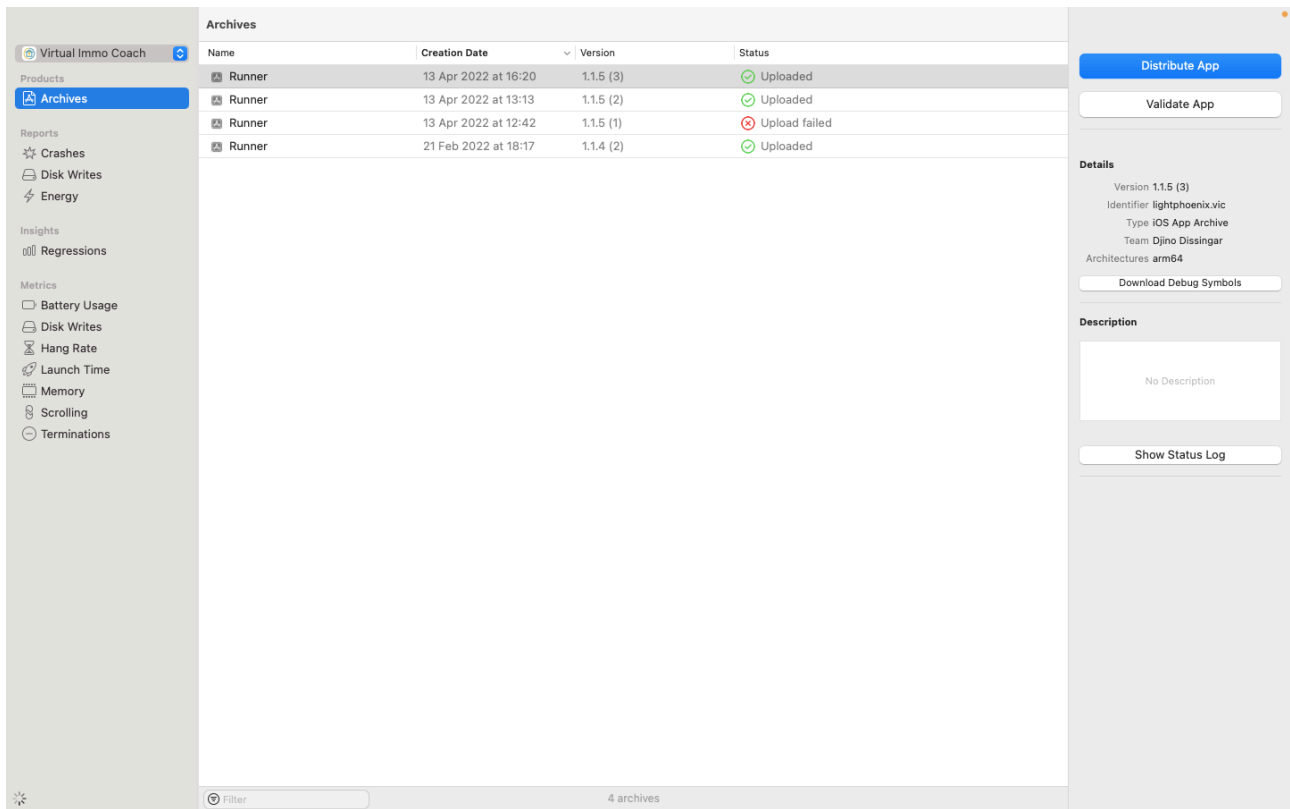
Déploiement sur les stores

Le déploiement sur les stores est la phase dans laquelle l'application sera compilé et envoyer sur les différents stores tel que Appstore, Playstore, AmazonStore etc.

Sous android, il y deux grandes phases, celle correspond à signer notre application pour des raisons de sécurité et d'unicité. Celle-ci se définit par la création d'une clé(key et d'une Keystore)

Voici une exemple pour créer une clé manuellement,

<https://docs.flutter.dev/deployment/android>



vous pouvez également créer votre clé via android Studio dans le menu Dans la barre de menus, cliquez sur Build > Generate un bundle signé/APK.

Une fois la création de la clé , il est nécessaire de construire votre fichier apk ou appBundle.

Dans Android Studio , celle ci est dans la boîte de dialogue> Build > Flutter > Build apk.

Une fois votre apk crée, il suffit d'uploader celui-ci sur le compte développeur Google.

Generate Signed Bundle or APK

Module

app

Key store path

~/user/keystores/upload-keystore.jks

Create new...

Choose existing...

Key store password

.....

Key alias

upload

Key password

.....

Remember passwords

Export encrypted key for enrolling published apps in [Google Play App Signing](#)

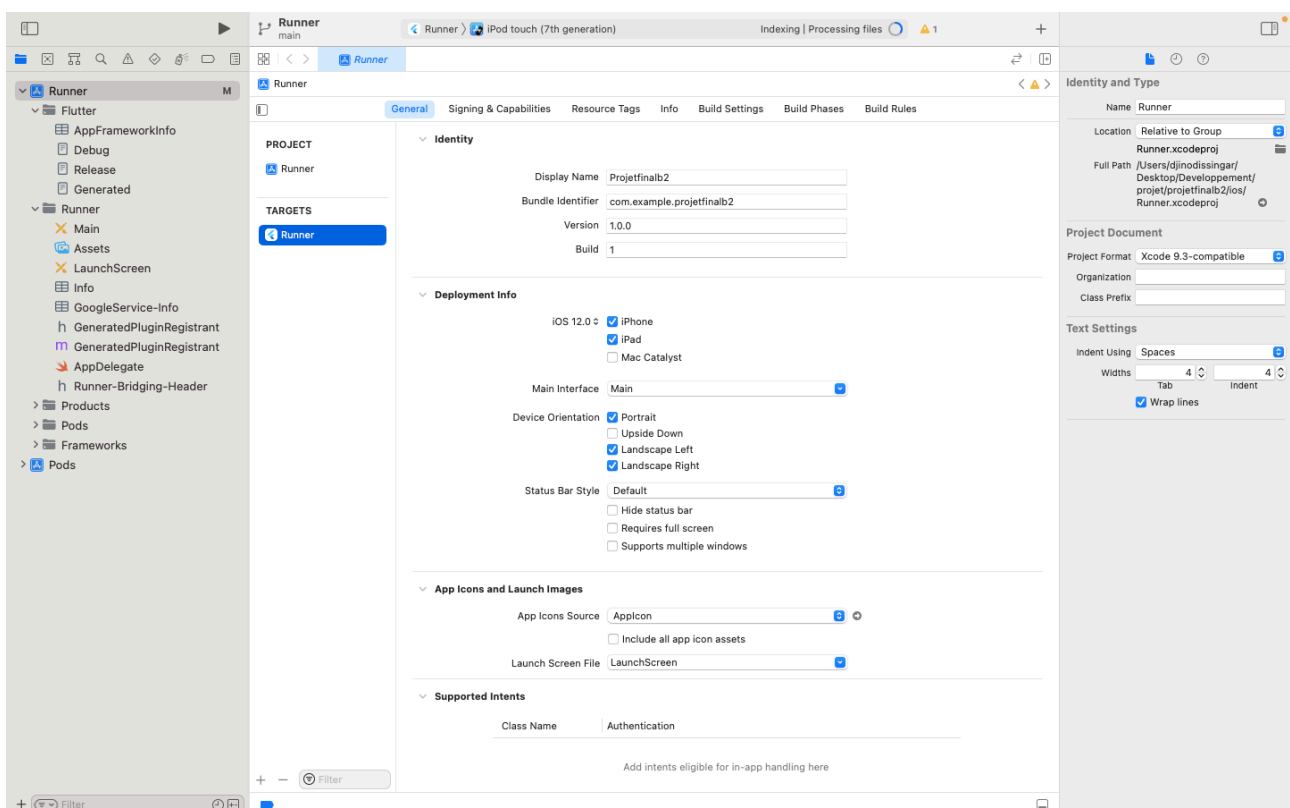
Help

Cancel

Previous

Next

Sous iOS, l'étape se déroule dans Xcode. Pour effectuer une compilation de votre programme allez dans le menu Product>Archive et indiquer également dans le menu Product>Destination> Any iOS device



Une fois la compilation , Xcode vous donne accès à l'archive il suffit de distribuer votre application sur le service de compte développeur.

Compte développeur

The screenshot shows the Apple App Store Connect interface for the app 'Africars Professionnel'. The browser address bar shows 'appstoreconnect.apple.com'. The app is listed under 'App iOS 1.0' with a status of '1.0 Prêt à la vente'. The left sidebar contains navigation links for 'App iOS', 'App macOS', and 'Ajouter une app tvOS'. The main content area is titled 'App iOS 1.0' and includes a section for 'Informations spécifiques à la version'. Below this, there are preview images for different device sizes: iPhone Affichage 6,5 po, iPhone Affichage 5,5 po, iPad Pro (3e génération) Affichage 12,9 po, and iPad Pro (2e génération) Affichage 12,9 po. The app's promotional text is visible at the bottom.

Africars Professionnel [App Store](#) [Services](#) [TestFlight](#) [Xcode Cloud](#)

App iOS 1.0 [Enregistrer](#)

App iOS [+](#)

1.0 Prêt à la vente

App macOS

1.0 Finaliser avant soumission

[Ajouter une app tvOS](#)

Général

- Informations sur l'app
- Tarifs et disponibilité
- Confidentialité de l'app
- Notes et avis
- Historique des mises à jour
- Vérification de l'app

Fonctionnalités

- Achats intégrés
- Promotions App Store
- Pages produit personnalisées
- Événements intégrés
- Optimisation de la page produit

Informations spécifiques à la version [Français ?](#)

La page produit de cette version d'app sera publiée dans l'App Store avec les ressources et métadonnées ci-dessous.

[Aperçus d'app et captures d'écran ?](#) [Afficher toutes les tailles dans le gestionnaire des visuels](#)

iPhone Affichage 6,5 po **iPhone Affichage 5,5 po** **iPad Pro (3e génération) Affichage 12,9 po** **iPad Pro (2e génération) Affichage 12,9 po**

Texte promotionnel [? Modifier](#)

Une fois, votre application présent sur les serveurs d'apple ou Google. L'étape suivante est de soumettre votre application à vérification auprès des différents services si cela est positif, vous pouvez mettre votre application sur les stores, dans le cas contraire il faut reconstruire un nouveau fichier et effectuer à nouveau les étapes à partir du build>Flutter> build apk ou Project>archive respectivement pour android et Apple.

Google Play Console

Recherchez des contenus sur la Play Console

Catalogue d'appareils

Appareils compatibles

17 716 modèles compatibles

Gérer les appareils

Exporter la liste des appareils

Modèle de l'appareil	Nom commercial	Versions Android	Mémoire RAM	Système sur une puce
10or 10or_D	10.or D	8.1	- (3 007 Mo)	Qualcomm MSM8917
10or E	10.or E	8.1	- (2 978 Mo)	Qualcomm MSM8937
10or G	10.or G	8.1	4 096 Mo (3 756 Mo)	Qualcomm MSM8953
10or G2	10.or 10or_G2	8.1	6 144 Mo (6 026 Mo)	Qualcomm SDM636
2E E450A2018	TW0E 2E E450A 2018	8.1	1 024 Mo (1 007 Mo)	Mediatek MT6580
2E E500A_2019	TW0E E500A_2019	8.1	1 024 Mo (1 000 Mo)	Mediatek MT6580
2E F534L	TW0E F534L	8.1	1 024 Mo (945 Mo)	Mediatek MT6739WA
2E F572L	TW0E F572L	8.1	2 048 Mo (1 970 Mo)	Mediatek MT6739WA

Conclusion

Flutter est également utilisé pour concevoir des applications avec des animations élaborés.

Aujourd'hui, Flutter est un Framework en devenir.

Beaucoup de projets sont actuellement développés sous Flutter (ex : Google Assistant environ 70% est sous Flutter , Gmail est en cours de transition pour être développé exclusivement sous Flutter)